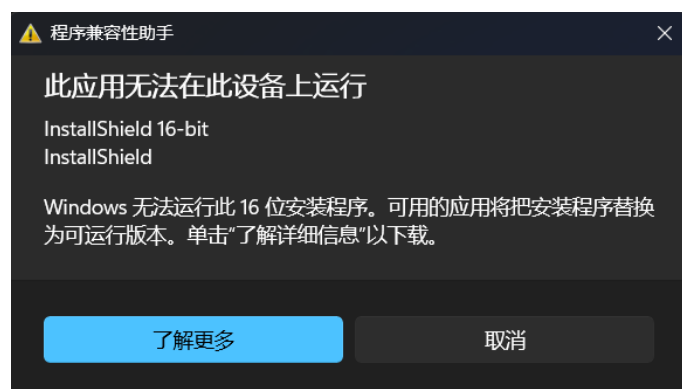


《网络与通信》课程实验报告

实验三：数据包结构分析

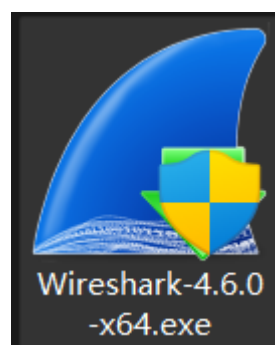
姓名	赵文字	院系	计算机学院	学号	23121769
任课教师	张瑞	指导教师	张瑞		
实验地点	计算机大楼 706	实验时间	2025/11/19		
实验课表现	出勤、表现得分(10)		实验报告 得分(40)	实验总分	
	操作结果得分(50)				
实验目的：					
1. 了解 Sniffer 的工作原理，掌握 Sniffer 抓包、记录和分析数据包的方法； 2. 在这个实验中，你将使用抓包软件捕获数据包，并通过数据包分析每一层协议。					
实验内容：					
使用抓包软件捕获数据包，并通过数据包分析每一层协议。					
实验要求：（学生对预习要求的回答）（10 分）					得分：
● 常用的抓包工具 Wireshark： 是一款非常流行的网络协议分析工具。可以捕获和分析网络上的各种数据包，支持多种协议的解析。 具有强大的过滤功能，能够根据特定的协议、IP 地址、端口号等条件进行数据包筛选。提供详细的数据包内容展示，包括数据包的头部信息、载荷数据等。 Fiddler： 主要用于 HTTP/HTTPS 协议的抓包。可以模拟客户端请求和服务器响应，方便进行测试和调试。支持断点调试，能够修改请求和响应的数据。提供丰富的插件扩展功能。 Charles： 功能与 Fiddler 类似，也是一款强大的 HTTP/HTTPS 抓包工具。具有良好的界面和易用性。支持 SSL 代理，可以解密 HTTPS 流量。可以进行流量限速，模拟不同网络环境下的应用性能。 Sniffer Pro： 是一款经典的商业网络协议分析软件。提供网络流量实时监控和统计功能，帮助管理员了解网络健康状况。具备强大的专家分析系统，可自动诊断常见的网络故障。支持深度解码多种网络协议，适用于企业级网络性能管理。 tcpdump： 是Linux/Unix系统上最基础、最强大的命令行抓包工具。在无图形界面的服务器环境中是网络排查的首选工具。通过灵活的过滤表达式精准捕获流量，输出结果可保存为文件供后续分析。资源占用极低，易于集成到自动化脚本中实现监控。 Burp Suite： 是一款专注于Web应用程序安全测试的集成平台。核心功能是拦截代理，可捕获、查看和修改浏览器与服务器之间的HTTP/HTTPS通信。提供重放、扫描器等模块，用于测试和验证Web应用漏洞。是进行Web安全渗透测试和漏洞分析的核心工具。 Ethereal： 是著名网络分析软件Wireshark的原始名称。作为Wireshark的前身，它奠定了现代开源协议分析器的基础。现已完全被功能更强大的Wireshark所取代。 NetXray / Iris： 是早期的图形化网络协议分析工具。					
实验过程中遇到的问题如何解决的？（10 分）					得分：

问题 1：在安装 sniffer Pro 的过程中，发生如下报错：“Windows 无法运行此 16 位的安装程序。”



因为我的电脑是 windows11 系统，是一个纯 64 位的操作系统，彻底移除了对 16 位应用程序的运行支持。而 sniffer Pro 是一个 16 位的应用程序。所以会提示无法兼容，无法在此电脑上运行 sniffer Pro。

解决方法：安装了另外一款流量抓包软件：wireshark。后续实验用该软件代替 sniffer Pro。



问题 2：在 TCP 抓包中，出现了[RST, ACK]的标识符号

140...	3043.673798	10.85.88.199	20.189.173.2	TCP	54 55048 → 443 [RST, ACK] Seq=3776 Ack=7035 Win=0 Len=0
--------	-------------	--------------	--------------	-----	---

“[RST,ACK]”指的是在 TCP 通信过程中，一方发送了一个带有 RST（Reset，重置连接）标志位的数据包，并且这个数据包同时带有 ACK（Acknowledgment，确认）标志位。这通常意味着通信的一方想要异常终止当前的 TCP 连接。

经在网上查阅了资料，发现原因可能有以下这三点：

第一点是连接建立异常：比如端口未监听，即客户端尝试连接到服务器的某个端口，但服务器上该端口没有程序在监听。例如，服务器上的应用程序未正确启动或配置，导致指定端口没有处于监听状态，服务器收到连接请求后发现无法处理，就可能发送“TCP RST ACK”数据包来终止连接；SYN 攻击防范：在服务器受到大量的 SYN（Synchronize，同步）请求攻击时，为了防止资源耗尽，服务器可能会对一些看起来可疑的连接请求直接发送“TCP RST ACK”来快速关闭连接。

第二点是网络环境问题，即网络设备故障：如路由器、交换机等网络设备出现故障，可能会导致数据包的转发异常，使通信双方的连接状态出现混乱，从而触发“TCP RST ACK”。例如，路由器的内存不足或 CPU 过载，导致无法正确处理数据包的转发；或是网络地址转换（NAT）问题：在使用 NAT 的网络环境中，如果 NAT 设备的配置错误或出现故障，可能会导致数据包的源地址或目的地址被错误地转换，从而使通信双方的连接出现问题，引发“TCP RST ACK”。

第三点则是与安全相关的原因，即防火墙拦截：防火墙可能会根据预设的规则拦截某些

TCP 连接，如果防火墙认为某个连接存在安全风险，可能会发送“TCP RST ACK”来中断连接。例如，防火墙检测到某个连接的数据包中包含可疑的内容或来自不信任的 IP 地址。或中间人攻击：在存在中间人攻击的情况下，攻击者可能会篡改数据包或干扰连接，导致通信双方的连接状态异常，从而引发“TCP RST ACK”。不过这种情况相对较少见，且需要特定的网络环境和攻击手段。

问题 3：存在两点相同的端口从属于一个数据包。

491	75.369036	61.151.230.185	10.85.88.199	TCP	62 80 → 61842 [RST] Seq=1 Win=0 Len=0
492	75.393440	61.151.230.185	10.85.88.199	TCP	62 80 → 61843 [RST] Seq=1 Win=0 Len=0

原因（此处展示的是最可能的原因）：端口映射错误或冲突：NAT 设备通常会维护一个端口映射表，记录内部设备的端口与外部端口的对应关系。如果端口映射表出现错误，或者多个内部连接的端口映射发生冲突，就可能导致同一个数据包被错误地发送到两个不同的目的端口。其他的原因还有但不限于负载均衡器的配置出现问题、或者在分发请求时出现错误，网络故障或错误配置，恶意攻击或网络异常行为等等。

解决办法：主机发送失败会自动进行重发，无需人工操作干预。

本次实验的体会（结论）（10 分）

得分：

在理论学习中，TCP/IP 四层模型、帧结构、IP 数据报、TCP/UDP 报文等概念始终是书本上的图表和文字，显得抽象而遥远。本次实验最大的收获，莫过于通过 Wireshark 实现了对这些概念的“可视化”解析。

当我在 Wireshark 界面中清晰地看到一个数据包被逐层分解为“Frame（物理帧）”、“Ethernet II（数据链路层）”、“Internet Protocol Version 4/6（网络层）”、“Transmission Control Protocol（传输层）”以及应用层协议（如 HTTP、DNS）时，那种豁然开朗的感觉是任何课本都无法给予的。我能够亲手点击展开每个协议头，观察诸如 MAC 地址、IP 地址、TTL 值、协议类型、源/目的端口、序列号、标志位等每一个字段的真实取值。例如，在分析一次 Ping 操作（ICMP 协议）时，我亲眼见证了 ICMP Echo Request 和 Echo Reply 报文如何在 IP 数据报的承载下完成往返，并验证了 IP 头中“协议”字段为 1，正对应 ICMP 协议。这种将理论字段与真实数据一一对应的过程，极大地巩固和深化了我的理论知识，使协议栈从平面的概念变成了立体的、可交互的实体。

实验一开始，我便遇到了一个关键挑战：实验指导书推荐的 Sniffer Pro 因其 16 位架构无法在 64 位的 Windows 11 系统上运行。这并非一个简单的操作失误，而是一个在现代计算环境下必然会出现的兼容性问题。我没有纠结于如何“强行”安装旧软件，而是通过查阅资料，迅速将实验工具迁移至当前业界标准——Wireshark。

这一解决问题的过程本身就是一个宝贵的学习经历。它让我认识到，在技术领域，工具是不断迭代更新的，但核心的原理和方法论是相通的。重要的不是死记硬背某个特定软件的操作，而是掌握这类工具的共同核心功能：捕获、过滤、解析。在 Wireshark 中，我学会了如何选择正确的网络接口，如何使用捕获过滤器在抓包初期精简数据，以及如何运用强大且灵活的表达式进行显示过滤（如 `tcp.port == 443` 来筛选 HTTPS 流量）。特别是“Follow TCP Stream”功能，能完整重组一次 TCP 会话，让我清晰地看到 HTTP 请求与响应的全过程，这对于理解应用层协议的工作原理至关重要。这次工具的成功迁移，锻炼了我的信息检索能力、适应性以及举一反三的思维，这是比单纯完成实验步骤更重要的能力提升。

通过对捕获数据包的细致分析，我得以窥见网络通信中许多隐藏的细节。例如，在思考题 1 中，我注意到一个编号为 4363 的数据包是一个 TCP 重传包。这引发了我的深入思考：为什么会出现重传？这可能是由于网络拥塞、丢包或对方响应超时所致。通过观察其序列号

和标志位（SYN），我判断出这是 TCP 三次握手第一次尝试的重复，说明客户端在首次尝试建立连接时未及时收到服务器的 SYN-ACK 确认。

此外，我还观察到了 IPv6 地址的广泛应用（如 2001:da8:8006:56000::* 与 Google DNS 的通信），这让我意识到下一代互联网协议正在成为现实。我也尝试捕获并分析了 DNS 查询、HTTP GET 请求等常见网络行为，每一次成功的解码都像是一次解谜，让我对日常使用的网络服务背后的复杂交互有了更深刻的敬畏之心。这种从庞杂的数据流中提取关键信息、发现问题线索并进行分析推理的能力，是网络故障诊断、安全分析和性能优化等高级工作的基础。总而言之，本次数据包结构分析实验是一次极具价值的实践环节。它成功地将《网络与通信》课程的理论知识落地，让我不再是理论的被动接受者，而是成为了网络世界的主动探索者。通过亲手捕获和分析数据包，我不仅牢固掌握了协议细节，更在实践中提升了工具使用、问题分析和解决问题的能力。

思考题：（10 分）

思考题 1：（4 分）

得分：

写出捕获的数据包格式。

从左到右捕获的数据包内容依次为：

捕捉编号 | 捕捉时间 | 源地址 | 目的地址 | 协议类型 | 数据长度 | 简单的数据信息

No.	Time	Source	Destination	Protocol	Length	Info
4352	5.937383	59.79.4.102	220.196.157.17	UDP	122	52782 → 8004 Len=80
4353	5.937666	59.79.4.102	220.196.157.17	UDP	138	52782 → 8004 Len=96
4354	5.954485	220.196.157.17	59.79.4.102	UDP	122	8004 → 52782 Len=80
4355	6.004885	175.27.3.92	59.79.4.102	RTCP	114	Receiver Report Application specific (loss) subtype=4Extended report (RFC 3611)
4356	6.048526	2001:da8:8006:3600::	2001:4860:4860::8888	TCP	86	58872 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1440 WS=256 SACK_PERM
4357	6.069490	52.203.195.129	59.79.0.9	TCP	60	443 → 58299 [FIN, ACK] Seq=1 Ack=1 Win=106 Len=0
4358	6.081232	59.79.4.102	8.8.4.4	TCP	66	58873 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
4359	6.111373	2001:da8:8006:3600::	2001:4860:4860::8888	TCP	86	[TCP Retransmission] 64601 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1440 WS=256 SACK_PERM
4360	6.218434	175.27.3.92	59.79.4.102	RTCP	114	Receiver Report Application specific (loss) subtype=4Extended report (RFC 3611)
4361	6.305976	175.27.3.92	59.79.4.102	RTCP	114	Receiver Report Application specific (loss) subtype=4Extended report (RFC 3611)
4362	6.350287	59.79.4.102	8.8.4.4	TCP	66	58874 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
4363	6.381750	2001:da8:8006:3600::	2001:4860:4860::8888	TCP	86	[TCP Retransmission] 58800 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1440 WS=256 SACK_PERM
4364	6.429460	59.79.4.102	8.8.4.4	TCP	66	[TCP Retransmission] 57043 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
4365	6.516456	175.27.3.92	59.79.4.102	RTCP	114	Receiver Report Application specific (loss) subtype=4Extended report (RFC 3611)
4366	6.568669	52.1.117.125	59.79.0.9	TCP	60	443 → 50098 [FIN, ACK] Seq=1 Ack=1 Win=106 Len=0
4367	6.623739	36.155.163.25	59.79.4.102	TCP	366	80 → 57269 [PSH, ACK] Seq=6953 Ack=1 Win=1889 Len=312
4368	6.667845	59.79.4.102	36.155.163.25	TCP	54	57269 → 80 [ACK] Seq=1 Ack=7265 Win=510 Len=0
4369	6.683361	36.155.163.25	59.79.4.102	TCP	454	80 → 57269 [PSH, ACK] Seq=7265 Ack=1 Win=1889 Len=400
4370	6.686929	59.79.4.102	36.155.163.25	TCP	746	57269 → 80 [PSH, ACK] Seq=1 Ack=7665 Win=509 Len=692

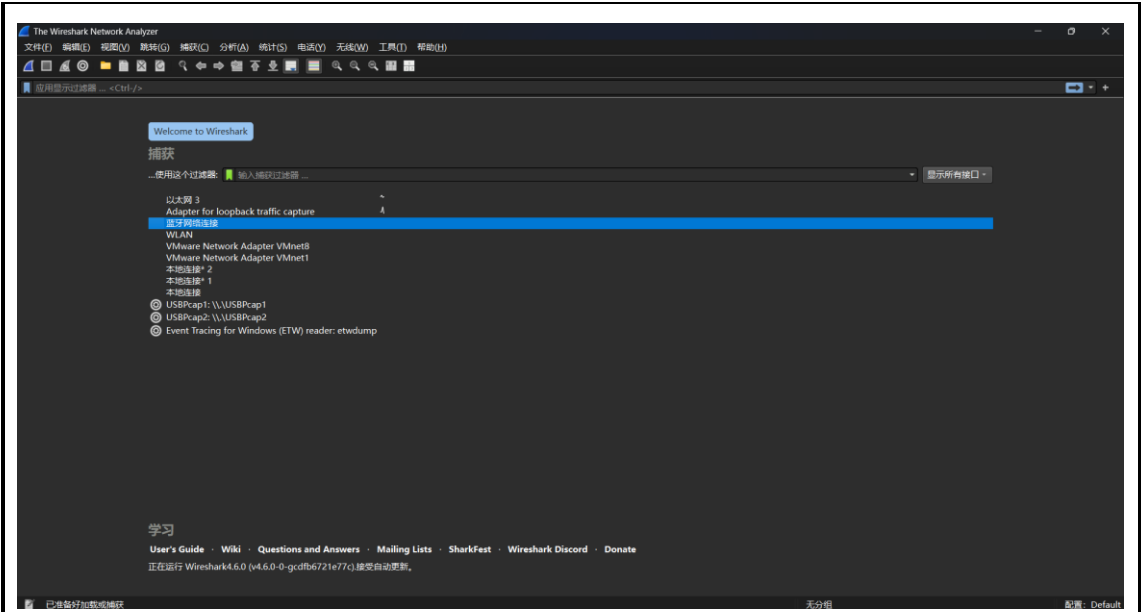
举上图编号为 4363 的那一行为例，捕捉编号为 4363，捕捉时间为 6.381750 秒，源地址为 IPv6 地址 2001:da8:8006:56000::*，目的地址为 IPv6 地址 2001:4560:4860:8888::*（这是一个 Google 公共 DNS 服务器），协议类型为 TCP，数据长度为 86 字节。简单的数据信息为：这是一个 TCP 重传数据包，由客户端临时端口 58800 发往 HTTPS 服务端口 443，用于尝试建立 TCP 连接的第一次握手（SYN），其序列号为 0。

思考题2：（6分）

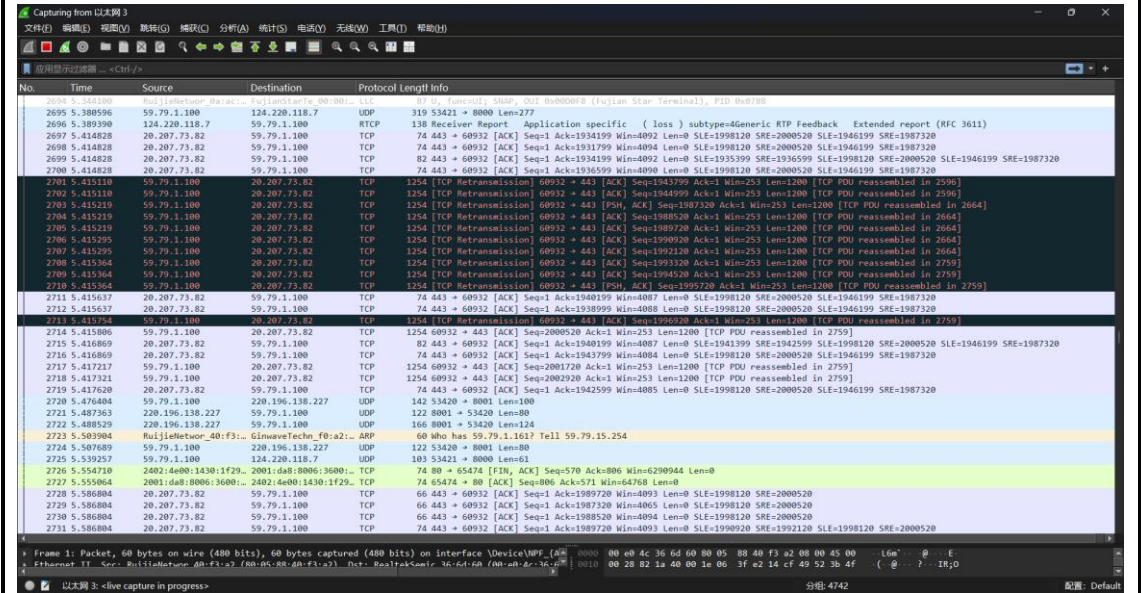
得分：

写出实验过程并分析实验结果。

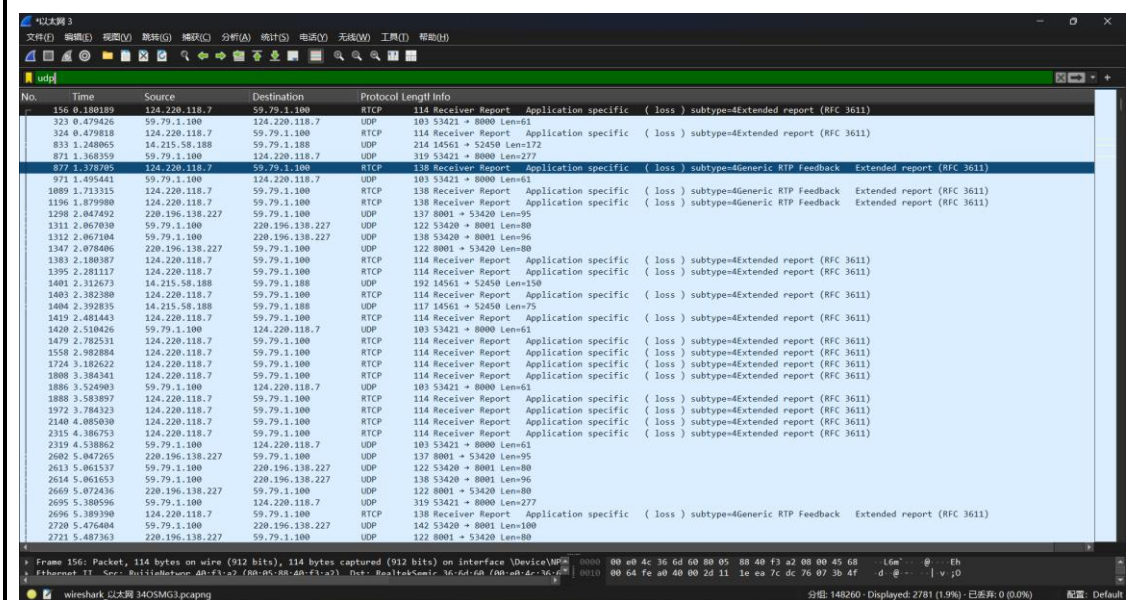
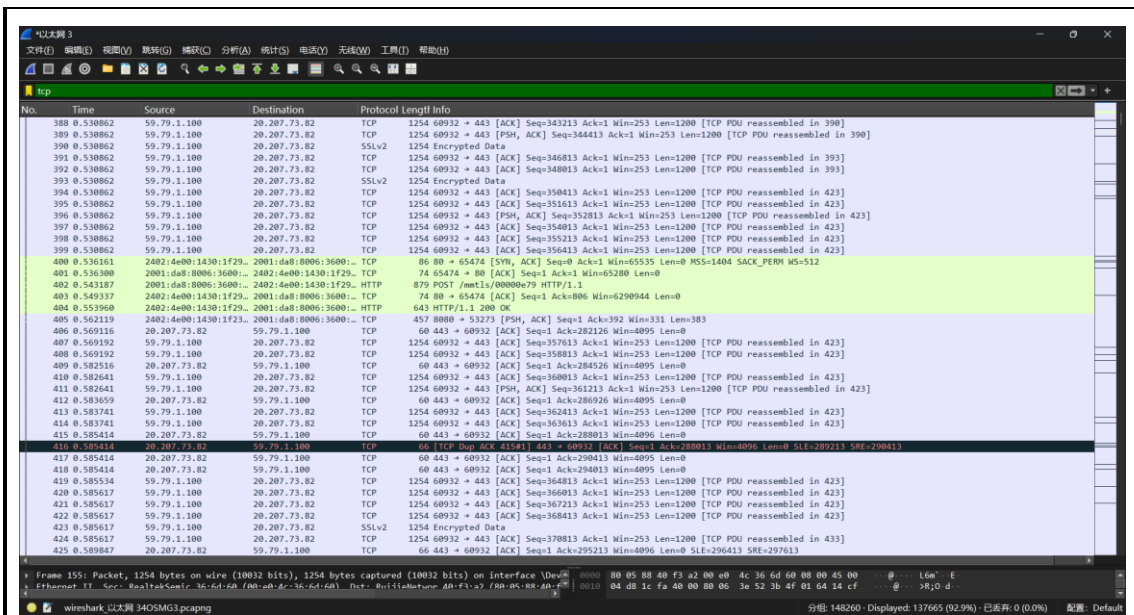
安装wireshark后打开该软件，进入主界面（如下图所示）：



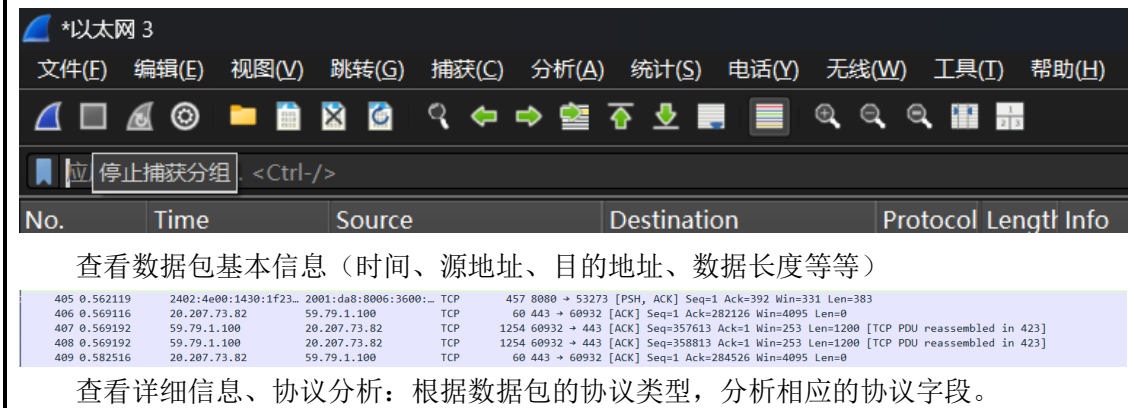
选择接口进行抓取数据包（本次实验抓取以太网3）

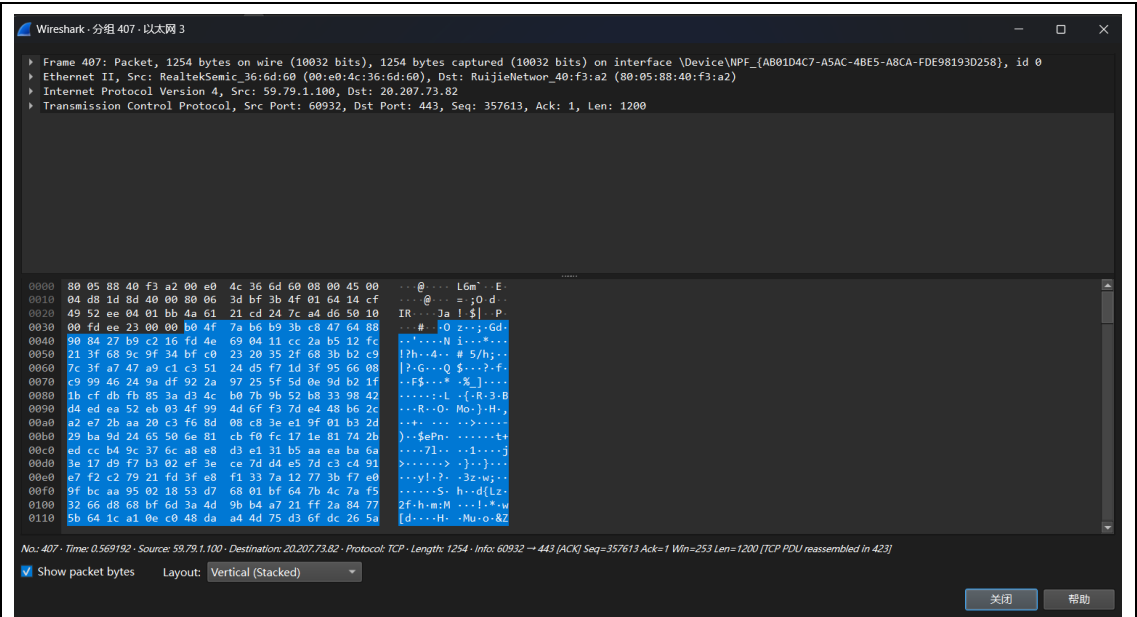


采用过滤（这里分别筛选了tcp与udp，如下图分别所示）：



停止抓包：





测试结束后关闭程序。

由于wireshark无法自行发送报文，所以无奈只能跳过该步骤。

指导教师评语：

日期：